

# Luxe Stay 酒店预订与信用管理平台

## 系统详细需求与架构设计说明书

### 第二部分：系统物理架构、业务模块与接口设计

## 一、引言

### 1.1 系统定义与解决问题

本系统命名为 **Luxe Stay 酒店在线预订与信用管理平台**，专为中高端酒店的线上直销与会员信用度量提供服务。系统主要解决传统平台在遭遇高频退订时房源库存白白锁定流失的痛点，创新性地提出“违约三次限期禁订”的信用管理规则；同时利用数据库本地事务级并发防护和价格快照设计，保障了酒店交易链条的高可靠与高一致性。

### 1.2 参考文献清单

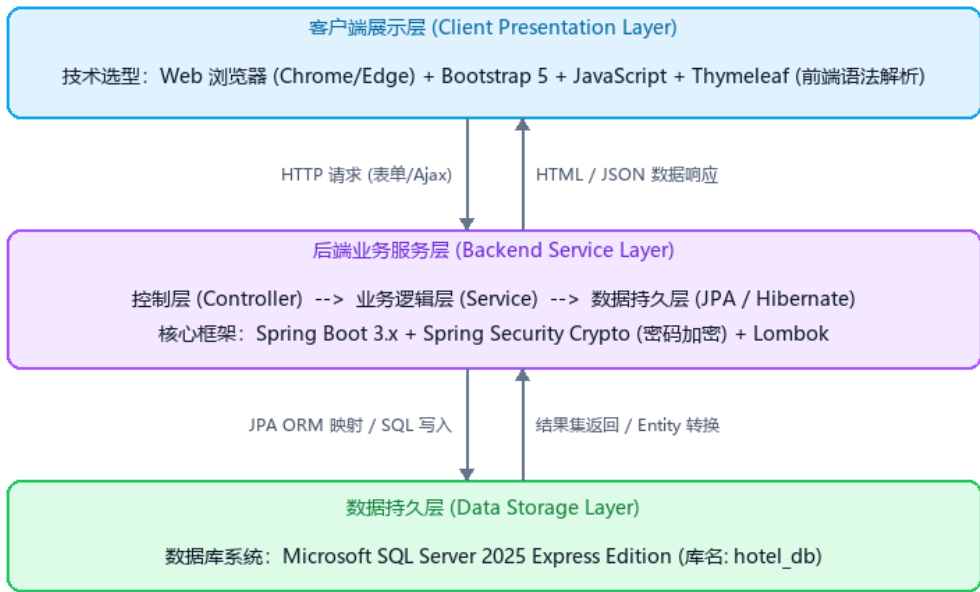
- [1] 酒店项目开发起步与 Git 协作指南.md, 本地开发指南
- [2] AI 草拟开发方向.md, 系统数据结构与业务规则规约
- [3] Craig Walls. Spring Boot in Action (4th Edition). Manning Publications, 2016.
- [4] Hibernate ORM 7.4.1.Final Official User Guide. Red Hat, Inc., 2025.
- [5] Microsoft SQL Server 2025 Database Engine Technical Documentation. Microsoft, 2025.

## 二、系统架构设计

### 2.1 分层架构关系说明

本平台基于典型的三层逻辑架构（物理层自上而下贯通）进行构建，具体分层系统架构图如下：

Luxe Stay 平台分层系统架构图



2.2 技术选型细化清单

根据项目的极简开发及易部署要求，平台的核心技术栈选型如下表所示：

| 架构层级  | 核心技术选型                                         | 作用描述与设计目的                               |
|-------|------------------------------------------------|-----------------------------------------|
| 开发语言  | Java 17 (LTS) / JavaScript (ES6) / SQL (T-SQL) | 保证强类型、高并发运行效率；原生语法支持。                   |
| 前端展示层 | HTML5 + Thymeleaf + Bootstrap 5                | 不使用复杂的独立前端脚手架，以服务器渲染直接输出，降低学习与前后端联调成本。  |
| 后端业务层 | Spring Boot 3.x + Spring Data JPA + Hibernate  | 实现依赖注入、事务管理（@Transactional）与持久化 ORM 模型。 |
| 数据持久层 | Microsoft SQL Server 2025 Express              | 提供带有外键强物理约束、事务 ACID 保护与锁控制的关系型数据库支持。    |

2.3 物理部署节点分布

- **客户端展示层**：部署于最终旅客或管理员的终端 PC 浏览器，通过 HTTP 请求拉取页面并渲染页面脚本。
- **业务逻辑服务层**：部署于搭载了 JRE 17 环境的单台物理/云服务器（如 Tomcat 中运行的内置 JAR 包），占用 8080 端口。

- **数据存储层**：部署于专用的关系型数据库主机，本地开发环境通过 localhost:1433 进行 TCP 直连。

## 三、系统业务模块设计

### 3.1 模块职责及需求映射

系统划分为三个高度协同的业务包模块，详情如下：

- **用户与权限模块：**

**模块职责：**负责账户注册、密码 BCrypt 强哈希加密、身份鉴权登录、退出；全局 Session 安全上下文维护，基于角色对 /admin/\*\* 后台路径实施 100% 权限拦截与 403 页面分流。

**对应功能需求：**F01 (注册加密), F02 (登录验证), F03 (越权拦截), F10 (违约惩罚逻辑)

- **房型与库存模块：**

**模块职责：**负责管理员对房型数据字典进行 JPA CRUD 维护与图片配置；配置每日具体日期的库存数据；处理前台按未来一周入住时间段做可用库存的实时级联计算与筛选。

**对应功能需求：**F04 (房型 CRUD), F05 (每日库存配置), F06 (按日期筛选房型)

- **预订与订单模块：**

**模块职责：**负责生成预订订单记录、逐日校验库存余量、触发 `@Transactional` 事务保证扣减原子性、记录快照单价防后期调价漏洞，以及在入住日至少一天前取消订单并原路释放回滚库存。

**对应功能需求：**F07 (跨天库存扣备), F08 (价格快照), F09 (取消退还库存)

### 3.2 模块间协作与调用关系

**1. 拦截器优先阻断：**当浏览器请求发出时，`用户与权限模块`的拦截器将首先被激发，判断 Session 会话状态。若尝试越权或未登录，直接完成跳转阻断，不流入后续业务模块。

**2. 预订信用前置检测：**当用户在`预订与订单模块`下单时，会先通过本地依赖注入调用`用户与权限模块`的`isUserBanned`服务。若用户有 3 次取消订单前科且未满 7 天禁订期，拒绝订单生成请求。

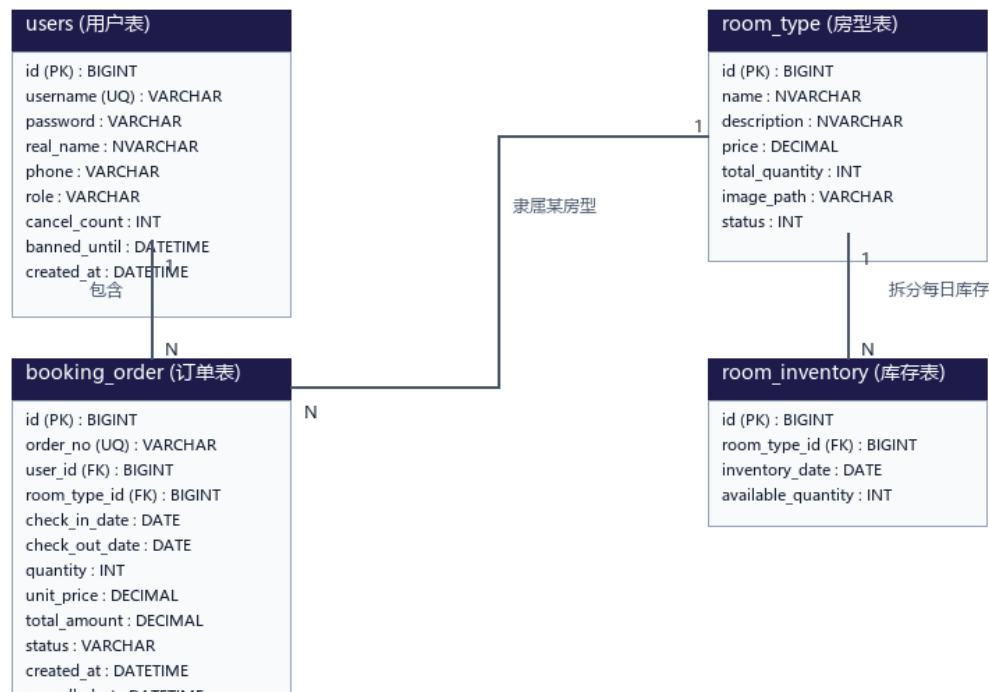
**3. 库存级联扣减与归还：**`预订与订单模块`在收到有效的预订与取消申请时，均以本地调用形式调用`房型与库存模块`的服务，完成每日库存的逐日核减与原子级回滚。

## 四、数据库逻辑与表结构设计

### 4.1 实体关系模型 (E-R Diagram)

本平台的数据实体间存在紧密的外键依赖与实体联系，核心表 E-R 关系图如下所示：

Luxe Stay 核心数据表 E-R 关系图



### 4.2 核心表结构物理细节清单

以下是数据库中四个核心关联物理表结构的详细定义：

#### ■ 表名：users (用户与信用表)

- id: BIGINT IDENTITY(1,1) PRIMARY KEY - 主键自增
- username: VARCHAR(50) NOT NULL UNIQUE - 登录账号（唯一）
- password: VARCHAR(100) NOT NULL - BCrypt 哈希加密密码
- real\_name: NVARCHAR(50) NOT NULL - 真实姓名
- phone: VARCHAR(20) NOT NULL - 联系电话
- role: VARCHAR(20) NOT NULL - 角色级别 ('USER' 或 'ADMIN')
- cancel\_count: INT NOT NULL DEFAULT 0 - 订单违约取消计数器
- banned\_until: DATETIME2 NULL - 预订权限冻结截止时间（NULL 表示正常）

#### ■ 表名：room\_type (房型字典表)

- id: BIGINT IDENTITY(1,1) PRIMARY KEY - 主键自增

- name: NVARCHAR(100) NOT NULL - 房型名称（如行政套房）
- description: NVARCHAR(500) NULL - 房型详情图文介绍
- price: DECIMAL(10,2) NOT NULL - 房型基准每晚单价
- total\_quantity: INT NOT NULL DEFAULT 0 - 房型配额房间总量
- image\_path: VARCHAR(255) NULL - 房型配图物理相对路径
- status: INT NOT NULL DEFAULT 1 - 启用标记 (1: 启用, 0: 禁用)

■ 表名：room\_inventory (每日库存表)

- id: BIGINT IDENTITY(1,1) PRIMARY KEY - 主键自增
- room\_type\_id: BIGINT NOT NULL (FK REFERENCES room\_type.id) - 房型外键
- inventory\_date: DATE NOT NULL - 具体库存日期 (精确到天)
- available\_quantity: INT NOT NULL DEFAULT 0 - 该日期可用空房剩余量
- UNIQUE (room\_type\_id, inventory\_date) - 联合唯一约束保证数据唯一性

■ 表名：booking\_order (预订订单表)

- id: BIGINT IDENTITY(1,1) PRIMARY KEY - 主键自增
- order\_no: VARCHAR(50) NOT NULL UNIQUE - 订单编号
- user\_id: BIGINT NOT NULL (FK REFERENCES users.id) - 用户外键
- room\_type\_id: BIGINT NOT NULL (FK REFERENCES room\_type.id) - 房型外键
- check\_in\_date: DATE NOT NULL - 入住起日
- check\_out\_date: DATE NOT NULL - 离店落日
- quantity: INT NOT NULL DEFAULT 1 - 预订间数
- unit\_price: DECIMAL(10,2) NOT NULL - 下单时刻单价快照（防止后续调价纠纷）
- total\_amount: DECIMAL(10,2) NOT NULL - 结算总付款金额
- status: VARCHAR(20) NOT NULL - 订单流状态 ('BOOKED', 'CANCELLED', 'COMPLETED')

## 五、系统接口设计 (API Design)

### 5.1 核心对外交互 API 清单

系统前后台对外暴露的核心服务路由如下表所示：

| 接口名称 | 请求方法 | 路由 URL | 功能描述     |
|------|------|--------|----------|
| 用户登录 | POST | /login | 验证账号密码，绑 |

|        |      |                      |                                                    |
|--------|------|----------------------|----------------------------------------------------|
| 用户注册   | POST | /register            | 定 Session 令牌并引导至对应主页。<br>校验用户名并使用 BCrypt 加密后存入数据库。 |
| 个人信息查看 | GET  | /profile             | 查询并展示会员信用计分、禁订期等基本信息。                              |
| 可用房型查询 | GET  | /rooms/search        | 基于入住离店日期区间实时剔除库存为 0 的房型。                           |
| 创建预订订单 | POST | /booking/create      | 逐日校验库存，并发抢占并写入预订订单（带事务控制）。                         |
| 取消预订订单 | POST | /booking/cancel/{id} | 原路返还房源库存，更新取消状态并累加信用取消计数。                          |

## 5.2 模块通信方式说明

- **前端与后端通信：**采用基于标准互联网传输协议的 **\*\*HTTP/1.1\*\*** 进行无状态通信。通过 Form 表单提交实现传统的页面切换导航，通过局部 Ajax 进行校验拦截等无感知交互。
- **后端模块间通信：**由于系统基于**\*\*单体物理架构\*\***开发，模块内各逻辑层（User、Room、Booking）全部运行于同一个 JVM 容器进程中，它们之间的通信完全由 Spring IOC 容器底层的 **\*\*Java 本地方法参数传递调用\*\***完成，模块间无需调用 RPC 服务，也无需走网络层，运行效率极高。

## 六、非功能需求实现方案

### 6.1 安全架构实现方案（加密与授权拦截）

**1. 密码学加固 (BCrypt)：**系统引入 `spring-security-crypto` 模块，注册时将明文密码输入 `BCryptPasswordEncoder`，经随机盐混淆计算产生 60 位长度的哈希密文后存入数据库。登录时调用 `matches` 方法比对，即使数据库的物理文件被拖库，明文密码也绝对无法被逆向推导破解。

**2. Spring MVC 拦截鉴权 (Session-Based Interceptor)：**在后端编写

[LoginInterceptor](file:///f:/CodexOutput/src/main/java/com/hotel/system/config/LoginInterceptor.java)，通过拦截器管道校验 `request.getSession().getAttribute("currentUser")`。未登录用户

访问受限资源将被统一重定向至 `/login` 页。普通用户角色（`USER`）若试图手动越权访问以 `/admin/\*\*` 开头的管理端控制路径，拦截器会自动识别并返回 `403` 无权访问的系统页面。

## 6.2 全局异常处理机制 (Global Exception Handling)

系统避免直接在 Controller 层使用冗长的 `try-catch` 代码污染业务，而是引入了统一的\*\*全局异常拦截切面\*\*。利用 Spring MVC 的 `@ControllerAdvice` 对所有业务层抛出的未捕获异常实施监听。

- **自定义异常类型：**系统根据异常性质定义了 `BannedException`（禁订期下单异常）和 `OutOfStockException`（库存不足异常）。
- **友好提示过滤：**当 Service 层检测到业务冲突并抛出自定义异常时，全局异常处理器捕获该异常，并将异常携带的友好业务文字提示回传至前端 Thymeleaf 页面，防止向客户端暴露敏感的后台代码运行栈报错。

## 6.3 数据库连接池与高并发吞吐保障 (HikariCP)

1. **高性能连接池调优：**系统默认集成了业内吞吐效率最高的 `HikariCP` 数据库连接池。通过在 `application.yml` 中配置连接池的生命周期参数（如 `minimum-idle: 5`、`maximum-pool-size: 15`），建立起长期稳定的 TCP 数据库复用通道，消除了大流量涌入时频繁 TCP 三次握手造成的连接延迟与服务器开销。
2. **并发安全与锁控制：**当多名用户在同一毫秒对最后剩余的一间空房进行抢订时，为了防止产生“超卖”现象，业务层在扣减库存时会使用物理排他锁或在 JPA 层面为库存实体添加乐观锁注解（`@Version`），使得同一时间仅有一个订单能扣减成功并提交事务，其余抢占失败的线程由于版本号失效自动触发异常回滚，从底层保证了库存数据的百分之百精确无误。